Práctica de Divide y Vencerás y Análisis de Algoritmos

Profesor de prácticas: Francisco Jose Montoya Dato

Miguel Ángel Tornel Martínez
Grupo 1, Subgrupo 2
mangel.tornel@um.es
DNI: 49971455E

Samuel Espín Santos
Grupo 1, Subgrupo 2
samuel.espins@um.es
DNI: 49476422H

Índice

1. Enunciado del problema **4**

1.1. Problema asignado 4

1.2. Descripción del problema 4

2. Diseño **4**

2.1. Diseño Iterativo 4

2.2. Diseño Divide y Vencerás 5

3. Implementación **6**

4. Validación **7**

4.1. Pruebas individuales 7

5. Análisis Teórico **8**

5.1. Definición del tamaño del problema 8

5.2. Tamaños de los subproblemas 8

5.3. Coste del algoritmo 9

5.4. Conclusión 9

6. Análisis experimental **10**

7. Contraste **10**

8. Conclusiones importantes **10**

1. Enunciado del problema

1.1. Problema asignado

Número de ejercicio: $49971455 + 49476422 = 99447877 \text{ mód } 10 + 1 = 8$

1.2. Descripción del problema

Dada una cadena A con n caracteres y un conjunto S de 5 caracteres distintos, devolver todas las subcadenas de A formadas por los 5 elementos de S sin repetir (permutaciones sin repetición). Habrá que obtener el número de subcadenas y su posición en la cadena A.

- Entrada: A (Cadena) y S (Conjunto)
- Salida: Subcadenas válidas y en que posiciones
- Ejemplo:
  - Entrada:
    - $A =$ `acbfabcedfcbcbadec`
    - $S = \{a, b, c, d, e\}$
  - Salida: 3 subcadenas válidas en las posiciones **5,13,14** (Empezando desde 1)

2. Diseño

En este apartado veremos el diseño de los dos algoritmos: Un algoritmo iterativo y un algoritmo basado en el divide y vencerás que se apoyará de la versión iterativa para casos atómicos que no se puedan dividir más y se deba resolver directamente mediante iterativo.

2.1. Diseño Iterativo

En este algoritmo se muestra cómo la versión iterativa resuelve el problema de contar las subcadenas y obtener las posiciones en las que empiezan. La idea es recorrer la cadena carácter a carácter. Para cada carácter comprobamos si pertenece al conjunto y lo guardamos en un historial de los últimos 5 caracteres. Esto nos permite comprobar si en ese grupo de caracteres se cumplen las condiciones del problema. El historial se mantiene utilizando una cola (FIFO). Cuando se añade un nuevo carácter y ya hay más de 5, se elimina el más antiguo. Mientras recorremos la cadena también llevamos un contador de cuántos caracteres distintos del conjunto aparecen en nuestro historial de 5 últimos caracteres. Si en algún momento ese número es igual a 5, significa que forman una subcadena válida, por lo que aumentamos el número de soluciones y guardamos su posición de inicio.

Esta versión iterativa también se utilizará para resolver algunos casos concretos dentro del algoritmo de Divide y Vencerás.

Algorithm 1 Número de subcadenas válidas y posiciones (iterativo)

```
function ITERATIVO(S, A, inicio, final)
    n ← final − inicio + 1
    letraContador ← diccionarioVacio
    historial ← colaVacia
    coincidencias ← 0
    numSubcadenas ← 0
    posiciones ← arrayVacio
    for i ← inicio to final do
        letra ← cadena[i]
        historial ← push(letra)
        if S ← contains(letra) then
            letraContador ← letraContador[letra] + 1
            if letraContador[letra] == 1 then
                coincidencias ← coincidencias + 1
            end if
        end if
        if historial ← size() > TAM_CONJUNTO then
            letraSale ← historial ← front()
            historial ← pop()
            if S ← contains(letraSale) then
                letraContador ← letraContador[letra] − 1
                if letraContador[letra] == 0 then
                    coincidencias ← coincidencias − 1
                end if
            end if
        end if
        if coincidencias == TAM_CONJUNTO then
            numSubcadenas ← numSubcadenas + 1
            posciones ← push(i − TAM_CONJUNTO + 2)
        end if
    end for
    return {numSubcadenas, posiciones}
end function
```

2.2. Diseño Divide y Vencerás

En este algoritmo se muestra una versión resumida de cómo se resolvería el problema usando la técnica de Divide y Vencerás. Primero se comprueban los casos base. Si la longitud de la cadena es menor que el tamaño del conjunto (5), entonces no puede haber ninguna solución. El otro caso base es cuando la longitud está entre 5 y 8. En este caso el problema se resuelve directamente con el algoritmo iterativo, ya que si dividiéramos una cadena tan pequeña en dos mitades obtendríamos subcadenas demasiado cortas para

contener soluciones. En el resto de casos el problema se divide en dos subproblemas: la mitad izquierda y la mitad derecha de la cadena. Se llama recursivamente al mismo procedimiento para resolver cada mitad. Después de resolver ambas partes, puede ocurrir que aparezcan nuevas soluciones en la frontera entre las dos mitades, es decir, subcadenas que empiezan en la izquierda y terminan en la derecha. Para comprobar estas posibles soluciones se analiza una pequeña zona alrededor de la división. Esta zona tiene como máximo longitud 8, por lo que se resuelve utilizando nuevamente el algoritmo iterativo. Finalmente se combinan las soluciones obtenidas en la parte izquierda, derecha y central para obtener el resultado final.

Algorithm 2 Número de subcadenas válidas y posiciones (Divide y Vencerás)

```
function DYV(S, A, inicio, final)
    n ← final − inicio + 1
    //Caso base 1
    if n < 5 then return {0, posicionesVacias}
    end if
    //Caso base 2
    if n ≤ 8 then return ITERATIVO(S, A, inicio, final)
    end if
    indexMitad ← inicio + (n/2)
    resIzq ← DYV(S, A, inicio, indexMitad − 1)
    resDer ← DYV(S, A, indexMitad, final)
    resCentro ← ITERATIVO(S, A, indexMitad − 4, indexMitad + 3)
    return MERGE(resIzq, resDer, resCentro)
end function
```

La función merge de sumar todas las subcadenas válidas que encontró izquierda, medio y derecha. Respecto a las posiciones de inicio de cada subcadena reutilizamos izquierda para insertar primero los de medio y luego los de derecha de esta manera tenemos la certeza que las posiciones iniciales de cada subcadena válida estan ordenadas.

Algorithm 3 Función auxiliar para combinar resultados en Divide y Vencerás

```
function MERGE(izq, der, cent)
    izq.numSubcadenas ← izq.numSubcadenas + der.numSubcadenas
    izq.numSubcadenas ← izq.numSubcadenas + cent.numSubcadenas
    izq.posiciones ← insert(izq.posiciones.end(), der.posiciones.begin(), der.posiciones.end())
    izq.posiciones ← insert(izq.posiciones.end(), cent.posiciones.begin(), cent.posiciones.end())
end function
```

3. Implementación

aaa

4. Validación

aaa

4.1. Pruebas individuales

Mejor caso $t_m(n)$:

- Entrada: `'zzzzzzzzzzzzzzzzzzzzzzzzzzzzzzz'`
- Conjunto: $\{a, b, c, d, e\}$
- Salida esperada (Iterativo): Nada
- Resultado (DyV): Nada

Peor caso $t_M(n)$:

- Entrada: `'abcdeabcdeabcdeabcdeabcdeabcdea'`
- Conjunto: $\{a, b, c, d, e\}$
- Salida esperada (Iterativo): 1 2 3 ... 27
- Resultado (DyV): 1 2 3 ... 27

Otro conjunto:

- Entrada: `'lkmnacbfadvxsa'`
- Conjunto: $\{c, b, f, a, d\}$
- Salida esperada (Iterativo): 6
- Resultado (DyV): 6

Cadenas menores de tamaño 5:

- Entrada: `'men'`
- Conjunto: $\{a, b, c, d, e\}$
- Salida esperada (Iterativo): Nada
- Resultado (DyV): Nada

5. Análisis Teórico

5.1. Definición del tamaño del problema

Sea el tamaño del problema:

$$n = final - inicio + 1$$

Si dividimos la cadena en dos subproblemas obtenemos que:

$$indexMitad = inicio + \left\lfloor \frac{n}{2} \right\rfloor$$

Entonces los subproblemas son:

- **Izquierdo:** $[inicio, indexMitad - 1]$
- **Derecho:** $[indexMitad, final]$

5.2. Tamaños de los subproblemas

Para el primer subproblema tenemos que:

$$n_1 = (indexMitad - 1) - inicio + 1 = indexMitad - inicio$$

Si sustituimos $indexMitad$ por su correspondiente valor obtenemos que:

$$n_1 = indexMitad - inicio = inicio + \left\lfloor \frac{n}{2} \right\rfloor - inicio = \left\lfloor \frac{n}{2} \right\rfloor$$

Para el segundo subproblema tenemos que:

$$n_2 = final - indexMitad + 1$$

Si sustituimos $indexMitad$ por su correspondiente valor obtenemos que:

$$n_2 = final - indexMitad + 1 = final - \left(inicio + \left\lfloor \frac{n}{2} \right\rfloor\right) + 1$$

$$final - \left(inicio + \left\lfloor \frac{n}{2} \right\rfloor\right) + 1 = final - inicio + 1 - \left\lfloor \frac{n}{2} \right\rfloor$$

$$final - inicio + 1 - \left\lfloor \frac{n}{2} \right\rfloor = n - \left\lfloor \frac{n}{2} \right\rfloor = \left\lceil \frac{n}{2} \right\rceil$$

Finalmente obtenemos que los tamaños de los subproblemas son $n_1 = \left\lfloor \frac{n}{2} \right\rfloor$ y $n_2 = \left\lceil \frac{n}{2} \right\rceil$

5.3. Coste del algoritmo

En consecuencia $t(n) = t(n_1) + t(n_2) + f(n)$ por tanto obtenemos:

$$t(n) = \begin{cases} C \in \Theta(1) & n < 5 \quad \text{(no hay subcadenas posibles)} \\ t_{it}(n) \in \Theta(n) & 5 \leq n \leq 8 \quad \text{(se resuelve iterativamente)} \\ t\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + t\left(\left\lceil \frac{n}{2} \right\rceil\right) + f(n) & n > 8 \quad \text{(divide y vencerás)} \end{cases}$$

Si asumimos que n es potencia de 2 ($n = 2^k$ para algún k entero). Bajo esta suposición, se tiene que

$$t(n) = \begin{cases} C \in \Theta(1) & n < 5 \quad \text{(no hay subcadenas posibles)} \\ t_{it}(n) \in \Theta(n) & 5 \leq n \leq 8 \quad \text{(se resuelve iterativamente)} \\ 2t\left(\frac{n}{2}\right) + f(n) & n > 8 \quad \text{(divide y vencerás)} \end{cases}$$

5.4. Conclusión

Como el problema se divide en 2 subproblemas del mismo tamaño tenemos $2t(\frac{n}{2})$. Por otro lado, $f(n)$ engloba todos los cálculos que se hacen en cada llamada a la recursión.
El **mejor caso** ocurre cuando ninguno de los caracteres de la cadena pertenece al conjunto S. En este escenario, el coste de `merge` $f(n)$ se mantiene constante, y la recurrencia del algoritmo sigue siendo

$$t(n) = 2\,t\left(\frac{n}{2}\right) + f(n)$$

Aplicando el Teorema Maestro se obtiene que

$$t_m(n) \in O(n).$$

El **peor caso** ocurre cuando todos los caracteres de la cadena pertenecen al conjunto S. En este caso, el coste de `merge` $f(n)$ es lineal en el tamaño de la cadena, pero la recurrencia sigue siendo la misma:

$$t(n) = 2\,t\left(\frac{n}{2}\right) + f(n)$$

Por lo que, aplicando nuevamente el Teorema Maestro, se obtiene que

$$t_M(n) \in O(n).$$

Hasta ahora estamos suponiendo que n es potencia de 2. Sin embargo, podemos generalizarlo para cualquier n usando el *teorema de la b-armónica*:

- $t(n)$ (con el techo y suelo) es eventualmente no decreciente.
- n es creciente.
- n es 2-armónica.

6. Análisis experimental

Si bien ya sabemos que el orden exacto teórico es n, lo cierto es que en la práctica si podemos establecer un mejor y un peor caso, el cual aumentará o reducirá la constante del merge. El mejor caso es cuando no se encuentra ninguna cadena, porque así no se debe realizar ningún tipo de combinación de resultados. El peor caso será pues lo contrario, cuando toda los haya n-4 coincidencias y por tanto tengamos que estar combinando los resultados todo el rato.

7. Contraste

8. Conclusiones importantes